



OSR eHSM 仿真环境说明

OSR IP

Copyright Notice and Proprietary Information Notice

Copyright © 2014 – 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市纽创信安科技发展有限公司保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技发展有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技发展有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

修订记录

版本号	生效日期	修改内容	编写人
V01	2024-07-26	Initial release	Fu-an.Xiao
V02	2026-01-22	更新部分参考文件名称	Jia-ze.Fang
V03	2026-05-12	增加 romcode 目录介绍章节	Jia-ze.Fang
V04	2026-05-18	同步最新 imgtool 工具生成 romcode 流程	Jia-ze.Fang

OSR CONFIDENTIAL

目录

目录	3
图片列表	4
表格列表	5
1 eHSM 仿真环境	6
2 Host 与 eHSM 交互方式	7
3 TB Case 运行过程	8
3.1 eHSM 仿真环境运行的初始化过程	8
3.1.1 SOC 初始化 OTP	8
3.1.2 SOC 软复位 eHSM	8
3.1.3 eHSM 硬件 Secure Boot 模块运行	8
3.1.4 eHSM CPU 开始运行	8
3.1.5 CPU 拉高 hsm_ready 信号	8
3.2 运行 Case	8
3.2.1 Host 将命令和数据写入 SOC_MEM	9
3.2.2 Host 将命令和结果地址合并(packet_addr)写入 SOC_MEM	9
3.2.3 Host 将 packet_addr 写入 eHSM 的 MB_S2H_INFO 寄存器	9
3.2.4 Host 写 Mailbox 的 MB_S2H_NOTE 寄存器触发中断	9
3.2.5 eHSM 从 MB_S2H_INFO 获知 cmd_addr 和 rsp_addr	10
3.2.6 eHSM 写 MB_S2H_NOTE 寄存器触发中断	10
3.2.7 eHSM 取命令和数据进行计算以及结果写入 SOC_MEM	10
3.2.8 eHSM 写 MB_H2S_NOTE 寄存器触发中断	10
3.2.9 Host 写 MB_H2S_NOTE 寄存器清除中断	11
3.2.10 Host 取计算结果判断是否计算正确	11
4 TB Case 说明	11
4.1 general_task 说明	11
4.2 hex 文件说明	13
4.3 ehsm_mbgen_tool-0.1 工具介绍	14
4.4 OSR 提供的 TB case 介绍	14
4.4.1 HASH TB case	14

4.4.2 SKE TB case	15
4.4.3 PKE TB case	16
4.4.4 TRNG TB case	17
4.4.5 扩展服务 TB case	17
4.4.6 PQC TB case.....	17
5 客户如何自己构造 case.....	18
6 romcode 使用介绍.....	21
6.1 imgtool 工具介绍	21
6.2 如何重新生成 romcode	22
6.3 生成 romcode 的参考流程.....	22

图片列表

Figure 1: SOC 与 eHSM 的交互方式.....	6
Figure 2: SOC_MEM 地址之间的关系	8
Figure 3: Hash 算法命令格式.....	9
Figure 4: Mailbox 波形示例 1	10
Figure 5: Mailbox 波形示例 2	10
Figure 6: Mailbox 波形示例 3	10
Figure 7: ehsm_hash_sha256_test (1 次 command packet 交互).....	11
Figure 8: ehsm_ske_sm4_test (2 次 command packet 交互)	11
Figure 9: general_task 说明.....	13
Figure 10: python 文件示例	18
Figure 11: hex 文件示例	19
Figure 12: 调用 general_task 构造 case 示例	19
Figure 13: 添加 case name 到 tb.sv 的位置.....	19
Figure 14: 添加 case 路径到 tc.sv 的位置.....	20

表格列表

Table 1: 上位机内存空间映射	6
Table 2: hex 文件数据含义	13

OSR CONFIDENTIAL

1 eHSM 仿真环境

eHSM 的 testbench 模拟了客户的 SOC 集成了 eHSM 之后的场景,即 SOC 芯片内集成了一个上位机(Host)和 eHSM 作为下位机,两者之间通过 Mailbox 进行通信,通过共享内存 SOC_MEM 进行数据交互,如图 1 所示。Host 可以通过 AHB 总线访问 eHSM 内部子模块的寄存器。除 Host 和 eHSM 共享的内存 SOC_MEM 之外,OTP 和 NVM 正常工作情况下不属于共享的存储区域,但是为了方便测试增加了后门,上位机也可以访问这些存储区域。上位机的操作,比如写入数据到共享内存 SOC_MEM,写 mailbox 寄存器等通过 task() 实现。

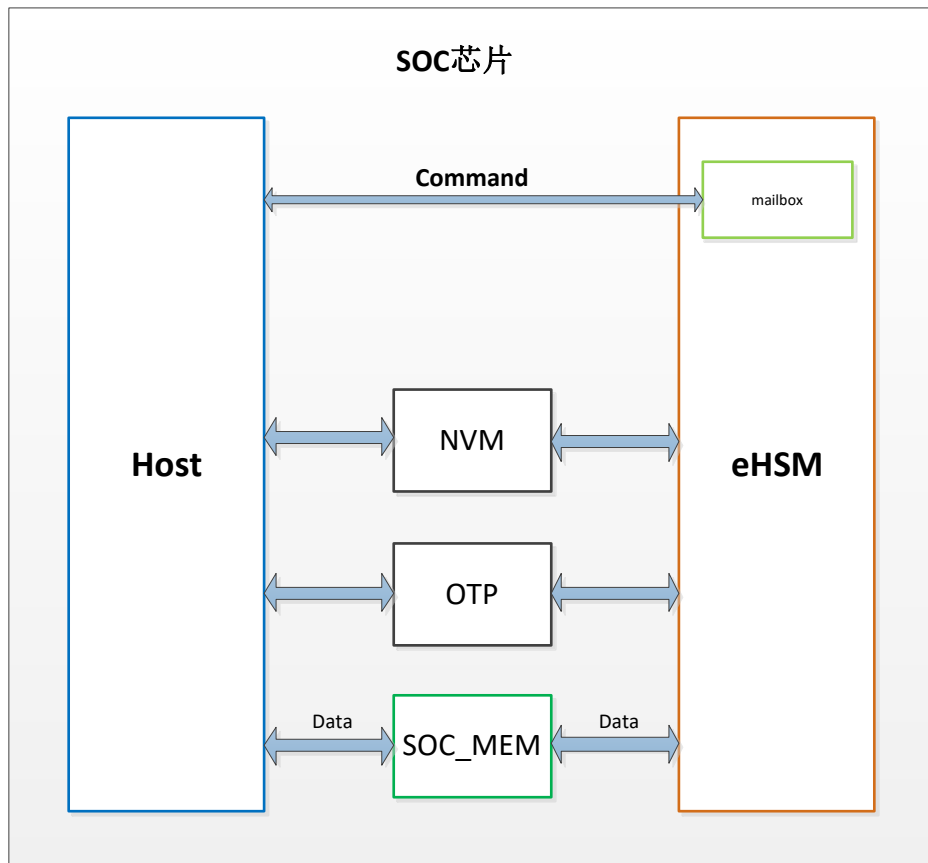


Figure 1: SOC 与 eHSM 的交互方式

上位机的内存空间映射如 Table1 所示,其中 SOC_MEM 是一块 host 和 ehsm 都可以访问的空间,host 将命令数据放到 SOC_MEM,然后通过 Mailbox 将基地址发送给 ehsm,ehsm 再去 SOC_MEM 取数据。仿真环境中的 OTP 和 IROM 都是用 RAM 实现,Host 访问 OTP 的地址相当于一个后门,方便初始化 OTP。

Table 1:上位机内存空间映射

Partitions	Boundary address	Blocks
IRAM 0x0000_0000 – 0x1FFF_FFFF	0x0000_0000 – 0x000F_FFFF	IRAM
	0x0010_0000 – 0x1FFF_FFFF	Reserved
DRAM 0x2000_0000 – 0x3FFF_FFFF	0x2000_0000 – 0x200F_FFFF	DRAM
	0x2010_0000 – 0x3FFF_FFFF	Reserved
AHB: 0x4000_0000 – 0x4FFF_FFFF	0x4000_0000 – 0x4000_FFFF	Reserved
	0x4001_0000 – 0x4001_FFFF	SYS
	0x4002_0000 – 0x4FFF_FFFF	Reserved

APB: 0x5000_0000 – 0x5FFF_FFFF	0x5000_0000 – 0x5000_3FFF	Reserved
	0x5000_4000 – 0x5000_4FFF	UART0
	0x5000_5000 – 0x5000_5FFF	UART1
	0x5000_6000 – 0x5FFF_FFFF	Reserved
SOC_MEM & OTP 0x6000_0000 – 0x7FFF_FFFF	0x6000_0000 – 0x6003_FFFF	SOC_MEM
	0x6004_0000 – 0x6007_FFFF	OTP
	0x6008_0000 – 0x7FFF_FFFF	Reserved
Mailbox 0x8000_0000 – 0x9FFF_FFFF	0x8000_0000 – 0x800F_FFFF	Mailbox
	0x8010_0000 – 0x801F_FFFF	NVM
	0x8020_0000 – 0x9FFF_FFFF	Reserved
Others	Others	Reserved

2 Host 与 eHSM 交互方式

Host CPU 通过 Mailbox 与 eHSM 进行命令交互，通过 SOC 与 eHSM 共享的 memory SOC_MEM 进行数据交互，实现 SOC 对 eHSM 的安全访问和安全操作，如密钥管理、签名验签、数据加解密等。Host 和 eHSM 之间的 Mailbox 通信协议及其交互流程，请参考 [OSR_eHSM_Firmware_TRM_vx.x.pdf](#) 的 Mailbox 通信协议章节。

Host 和 eHSM 之间的命令交互并不是直接将 command 通过 Mailbox 的 Registers 发送给 eHSM，而是先将 command 写入 SOC_MEM 的 cmd_addr 中，然后将 command 的首地址 cmd_addr 和存放计算结果的首地址 rsp_addr 也写入一个 packet_addr 中，最后将 packet_addr 通过 Mailbox 发送给 eHSM。

三个地址之间的关系如下：

cmd_addr: 存放 cmd_id、process_mode、algorithm 和 data 等 command 数据的首地址；

rsp_addr: 存放 eHSM 计算结束后输出的结果数据的首地址；

packet_addr: 存放 cmd_addr 和 rsp_addr 的首地址。

Host 将 packet_addr 通过 Mailbox 发送给 eHSM，eHSM 从 packet_addr 可以获知命令地址 cmd_addr 和结果写回地址 rsp_addr。

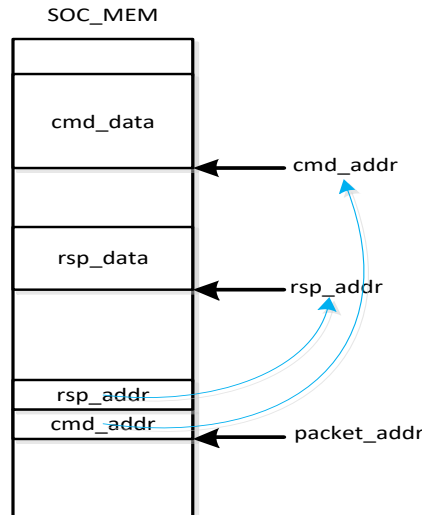


Figure 2: SOC_MEM 地址之间的关系

3 TB Case 运行过程

3.1 eHSM 仿真环境运行的初始化过程

eHSM 的仿真环境的运行过程包括以下步骤:

3.1.1 SOC 初始化 OTP

tb.sv 中调用 tc.sv 中的 task init_boot_data(), 完成 OTP 数据的初始化, OTP 的数据参考 TRM 中的 3.1.1 节 OTP Memory Map.

3.1.2 SOC 软复位 eHSM

初始化 OTP 结束后, SOC 软复位 eHSM;

3.1.3 eHSM 硬件 Secure Boot 模块运行

Boot 模块负责读 OTP 中的信息, 包括 life_cycle、HW_ctrl、FW_ctrl, 以及将 key 解密后存入 kmu ram 中, 解密成功后拉高 o_hw_boot_ok 信号;

3.1.4 eHSM CPU 开始运行

Boot 模块的 o_hw_boot_ok 信号拉高后释放 CPU 的复位, CPU 开始运行, 使能 mbox 和 random clock 的中断等配置;

3.1.5 CPU 拉高 hsm_ready 信号

eHSM 的软件完成初始化之后, 软件写 CSR SYS_STA0 寄存器, 拉高 hsm_ready 信号, 表示 eHSM 已经准备好进行 secure service;

3.2 运行 Case

eHSM 的软硬件完成初始化之后, 根据 make sim 时指定的 tc_case_name, 进入某个 case 开始运行, case 的运行过程如下。

3.2.1 Host 将命令和数据写入 SOC_MEM

Host 将包含 cmd_id、process_mode、algorithm 和 data 等 command 数据的一个数据包写入 SOC_MEM 中以 cmd_addr 为起始地址的空间中。下图是 Hash 算法的命令格式，其它算法的命令格式请参考 OSR_eHSM_Firmware_TRM_vx.x.pdf 的 Mailbox 命令章节。

表 10-14: CMD-HASH

Byte.idx	Name (little-endian)	Description
0-1	cmd_id	command id = 0x0301
2-3	cmd_id_inv	command id inverse
4-11	reserved0	must set to 0
12-12	process_mode	specifies the access mode refer to process-mode . other values are reserved.
13-14	reserved1	must set to 0
15-15	algorithm	Hash algorithm • 0x00: SM3 • 0x01: MD5 • 0x02: SHA256 • 0x03: SHA384 • 0x04: SHA512 • 0x05: SHA1 • 0x06: SHA224 • 0x07: SHA512_224 • 0x08: SHA512_256 • 0x09: SHA3_224 • 0x0a: SHA3_256 • 0x0b: SHA3_384 • 0x0c: SHA3_512
16-35	reserved2	must set to 0
36-43	data	Address of message data for hash generation.

Figure 3: Hash 算法命令格式

3.2.2 Host 将命令和结果地址合并(packet_addr)写入 SOC_MEM

Host 将 2 个 word 的 cmd_addr 和 2 个 word 的 rsp_addr，写入 SOC_MEM 中以 packet_addr 为起始地址的空间中。

3.2.3 Host 将 packet_addr 写入 eHSM 的 MB_S2H_INFO 寄存器

Host 将 packet_addr 写入 Mailbox 的 MB_S2H_INFO 寄存器中，如 figure 4 的 2 处所示。所有的 tb case 使用第一个 mailbox(起始为第 0 个)来进行通信，如 figure 4 的 1 处所示。

3.2.4 Host 写 Mailbox 的 MB_S2H_NOTE 寄存器触发中断

Host 写 1 置位 Mailbox 的 MB_S2H_NOTE 寄存器的 bit 0, 如 figure 4 的 3 处所示。触发 Mailbox 中断，通知 eHSM 有指令发送，如 figure 4 的 4 处所示。



eHSM 通过中断获知 Host 发送了指令，eHSM 从 MB_S2H_INFO 取到 `packet_addr`，如 figure 5 红框中所示，读 2 个 MB_S2H_INFO 寄存器，第 1 个 word 读到 `packet_addr`，第 2 个 word 读到为 0。随后 HSM 将从 `packet addr` 读取 `cmd addr` 和 `rsp addr`。



eHSM 写 1 清零 MB_S2H_NOTE 的 bit 0，如 figure 6 红框中所示。触发 Mailbox 中断，通知 Host 已经收到了指令。



eHSM 从 `cmd_addr` 取到 `cmd_id`、`process_mode`、`algorithm` 和 `data` 等数据，解析这些数据后调用算法模块进行计算。eHSM 计算完成后，将计算结果写入 SOC MEM 的 `rsp_addr` 中。

eHSM 写 1 置位 Mailbox 的 MB H2S NOTE 寄存器的 bit 0, 触发 Mailbox 中断, 通知 Host 作为计算结

束标志。

3.2.9 Host 写 MB_H2S_NOTE 寄存器清除中断

Host 通过中断获知 eHSM 处理完成标志。Host 写 1 清零 Mailbox 的 MB_H2S_NOTE 寄存器的 bit 0, 触发 Mailbox 中断, 通知 eHSM 已经收到了完成标志。

3.2.10 Host 取计算结果判断是否计算正确

Host 从 `rsp_addr` 中取出计算结果和返回码, 判断计算结果是否正确, 返回码为 32'ha55a 则表示计算正确, 否则计算错误。

4 TB Case 说明

交付件中提供了 tb case 示例, 详见 [sim/run_regression](#)

Case 路径: [sim/tc/case](#)

hex 文件路径: [sim/tc/case/hex](#)

4.1 general_task 说明

每个 case 中调用了一个通用的任务 `general_task()`, 调用一次 `general_task()` 就是一次 command packet 交互, 它的三个输入分别是: 指定要写入的 SOC_MEM 的一个地址(此处为 0x6000_0000), hex 文件路径和 case 名称, 一个输出是错误返回码 `rsp_code`。

有些 case 只需要 1 次 command packet 交互即可, 比如 `hash_sha256`、`trng_test`, 此外的 SKE 和 PKE 的 case 都需要先 import key, 然后再做 cipher 或者 verify 等, 所以需要 2 次 command packet 交互。

```

9 task ehsm_hash_sha256_test();
10 reg [31:0] key_import_return_handle;
11
12 general_task(64'h6000_0000, "../tc/case/hex/hash_sha256.hex", "hash_sha256", key_import_return_handle);
13
14 endtask

```

Figure 7: ehsm_hash_sha256_test (1 次 command packet 交互)

```

9 task ehsm_ske_sm4_test();
10 reg [31:0] key_import_return_handle;
11 //key_import
12 general_task(64'h6000_0000, "../tc/case/hex/import_sm4_key.hex", "import_sm4_key", key_import_return_handle);
13 //ske_calc
14 general_task(64'h6000_0000, "../tc/case/hex/sm4_cipher.hex", "sm4_cipher", key_import_return_handle);
15
16 endtask

```

Figure 8: ehsm_ske_sm4_test (2 次 command packet 交互)

参照 figure 9, `general_task()` 的功能如下:

1. `write hex_data`: 读取输入的 hex 文件内容并写入指定的 SOC_MEM 地址(此处为 0x6000_0000);

其中用到的 `task set_data_n()` 作用就是往给定地址写入一定长度的数据, 它的三个参数 `i_base_radix`、`i_world_len` 和 `i_data[$]` 分别指定写入数据的基地址, 要写入数据的世界长度, 以及待写入的数据队列。写入的地址从基地址递增, 依次写入 `i_data[$]` 中的数据。

2. `write TEST_PACKET_ADDR to s2h_info`: 将 hex 文件中包含的 `TEST_PACKET_ADDR` (相对基地址的 offset 为 32'hca0) 写入 Mailbox 的 MB_S2H_INFO 寄存器。其中 `MAILBOX_BASE` 是 SOC

端写 mailbox 时的基地址, 需要改成客户自己 SOC 端写 mailbox 时的基地址。MAILBOX_BASE 后加的 offset 64'h1000, 其中[15:12]表示选择第几个 mailbox, 这里是第 1 个; [11:0]是 mailbox 内部的寄存器地址, 参考 [OSR_eHSM_HP_Technical_Reference_Manual_CN.pdf](#) 的 Mailbox 章节;

其中用到的 task `upper_ahb_wr()`作用是上位机通过 AHB 总线往某个地址写入一个值, 每次只写一个地址, 它的两个参数 `waddr[63:0]`和 `wdata[31:0]`分别指定要写入数据的地址和要写入的数据。

3. `set s2h_note`: 写 1 置位 Mailbox 的 MB_S2H_NOTE 寄存器的 bit 0, 触发 Mailbox 中断, 通知 eHSM 有指令发送;
4. `read h2s_note`: 轮询 MB_H2S_NOTE 寄存器, 如果读到 32'h1, 则表示 eHSM 已经接收到 SOC 发送的 command 并且计算结束;

其中用到的 task `upper_ahb_rd()`作用是上位机通过 AHB 总线读取某一个地址的数据, 每次只读一个地址, 它的两个参数 `raddr[63:0]`和 `upper_reg_rdata[31:0]`分别指定要读取的地址和读取到的数据。

5. `write 1 to clear h2s_note`: 写 1 清零 MB_H2S_NOTE 的 bit 0, 表示 SOC 已经接收到 eHSM 计算结束的标志;
6. `get result`: 去 hex 文件中包含的 `TEST_EXPECT_DATA_ADDR` 地址 (此处固定为 0x6000_08a0) 读取 golden 结果, 一共读取 8 个 word (需要注意的是有些 command 没有 golden 参考数据, 比如 `import_key` 和 `trng_test` 时, 读回的 `golden_result` 为全 0); 去 hex 文件中包含的 `TEST_CHECK_DATA_ADDR` 地址 (此处固定为 0x6000_04a0) 读取实际计算输出结果, 一共读取 8 个 word;

其中用到的 task `get_result_n()`的作用是从一个基地址开始, 读取固定长度的数据, 通过队列输出。它的四个参数 `i_base_radix[63:0]`、`i_world_len[63:0]`、`o_result[$][31:0]`和 `name` 分别指定读取数据的基地址、读取数据的 world 长度、输出的队列和打印输出的字符串名称。

7. `get rsp`: 去 hex 文件中包含的 `TEST_RET_CODE_ADDR` 地址 (此处固定为 0x6000_0080) 读取返回码 `return_code`, 软件约定返回码为 32'hA55A 则表示计算结果正确, 否则计算错误。
8. 根据 `rsp_code`, tb 平台将打印输出 PASS 或 FAIL 字样, 分别表示 case pass 和 case fail。

```

13
14 $display("##### %s start #####\n",cmd_name);
15 //step_1
16 $display("-----write hex_data-----\n");
17 $readmemh(hex_file,hex_data);
18 set_data_n(hex_addr,812,hex_data); //write hex data to SOC_MEM
19
20 //step_2
21 $display("-----write TEST_PACKET_ADDR to s2h_info-----\n");
22 upper_ahb_wr(MAILBOX_BASE+64'h100,hex_addr[31:0]+32'hca0); //send TEST_PACKET_ADDR to Mailbox
23
24 //step_3
25 $display("-----set s2h_note-----\n");
26 upper_ahb_wr(MAILBOX_BASE+64'h100,32'h1); //write s2h_note, trigger Mbox irq
27
28 //step_4
29 $display("-----read h2s_note-----\n");
30 upper_ahb_rd(MAILBOX_BASE+64'h104,reg_rdata); //read h2s_note
31 while(reg_rdata[0] != 1'b1) begin
32     upper_ahb_rd(MAILBOX_BASE+64'h104,reg_rdata);
33 end
34 $display("h2s_note = %h",reg_rdata);
35
36 //step_5
37 $display("-----write 1 to clear h2s_note-----\n");
38 upper_ahb_wr(MAILBOX_BASE+64'h104,32'h1);
39
40 //step_6
41 $display("-----get result-----\n");
42 get_result_n(hex_addr+64'h8a0,8,golden_result,"golden_result");
43 get_result_n(hex_addr+64'h4a0,8,actual_result,"actual_result");
44
45 actual_result = {};
46 golden_result = {};
47
48 //step_7
49 $display("-----get rsp-----\n");
50 if(reg_rdata == 32'h1) begin
51     upper_ahb_rd(hex_addr+64'h80,rsp_code );
52     upper_ahb_rd(hex_addr+64'h84,rsp_data0);
53     upper_ahb_rd(hex_addr+64'h88,rsp_data1);
54 end

```

Figure 9: general_task 说明

4.2 hex 文件说明

所有 tb case 的 hex 文件数据量都是 812 个 word，所以每个 hex 文件都是 812 行，general_task.sv 中往 SOC_MEM 写 hex 时的 data_len 也固化为 812 个 word(见 general_task.sv #18 行)。以下介绍 hex 文件这 812 个 word 数据的含义：

Table 2: hex 文件数据含义

Offset	Length (byte)	Description
0x0000_0000~0x0000_007C	128	CMD , 每个算法的命令长度不同, 不足 128 byte 的填充 0。
0x0000_0080~0x0000_009C	32	Return code , 即返回码。全部为 0, 等 eHSM 计算结束后会将返回码回写到该地址。
0x0000_00A0~0x0000_0049C	1024	Data input value , 即输入数据, 不足 1024 byte 的填充 0。
0x0000_004A0~0x0000_0089C	1024	Check data addr , 放实际计算输出结果的地址。全部为 0, 等 eHSM 计算结束后会将

		计算结果写入该地址。
0x0000_008A0~0x0000_0C9C	1024	Expect data addr , 放 golden 参考输出结果的地址, 不足 1024 byte 的填充 0。Golden 的结果由软件密码工具计算得到。
0x0000_0CA0~0x0000_0CB0	16	Packet addr , 低 8 个 byte 放置的是 CMD 的首地址(0x0000_0000), 高 8 个 byte 放置的是 return code 的首地址(0x0000_0080)。Host 只需要将 Packet addr 的首地址通过 Mailbox 发送给 eHSM, eHSM 即可获得完整的命令格式数据。

注意:

1. 上述表格中只有地址 offset, 实际地址需要加上 SOC_MEM 的基地址, 交付件中 tb 平台中基地址为 0x6000_0000, **客户自己的 tb 平台需要将基地址改成自己的。**
2. Host 只将 Packet addr 的首地址通过 Mailbox 发送给 eHSM, eHSM 通过读取 Packet addr 的数据进而获得 **CMD** 和 **Return code** 的地址。**Data input value** 和 **Check data addr** 的首地址包含在 CMD 字段内, 所以 eHSM 可以从 CMD 内容中解析出 **Data input value** 和 **Check data addr** 的首地址。
3. **Expect data addr** 并不需要通知给 eHSM, 只是存放 golden 的输出结果, 可有可无。判断最终计算结果的正确与否不依赖 golden 输出结果和实际输出结果的比对, 而是依赖软件回写的 return code, 如果 return code 为 32'hA55A 则表示计算正确。

4.3 ehsm_mbgen_tool-0.1 工具介绍

ehsm_mbgen_tool-0.1 是一个测试数据生成工具, 用户只需要配置 python 文件, 然后运行脚本即可自动生成固定格式的 hex 文件。详细使用方法请查看/ehsm_mbgen_tool-0.1/文件夹下的 Readme.md。

4.4 OSR 提供的 TB case 介绍

OSR 提供以下 TB case, 覆盖所有的算法核(**注意: PQC TB case 只有集成了 PQC 模块时才提供**)。

4.4.1 HASH TB case

4.4.1.1 ehsm_hash_sha256_test

测试 HASH 算法命令, Host 配置以下命令结构体, 等待 eHSM 计算结束, 校验计算结果正确性。

- Cmd_id: 0x0301
- Process_Mode: one-pass
- Algorithm: SHA256

4.4.1.2 ehsm_hmac_sm3_test

测试 HMAC 算法命令, Host 配置以下命令结构体, 等待 eHSM 计算结束, 校验计算结果正确性。

- Cmd_id: 0x0302
- Process_Mode: one-pass

- MAC_Direction: Mac_gen
- Algorithm: SM3

4.4.2 SKE TB case

4.4.2.1 ehsm_ske_sm4_test

测试对称加解密算法命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0101
- Process_Mode: one-pass
- Cipher_Direction: Cipher_enc
- Algorithm: SM4
- Cipher_Mode: ECB

4.4.2.2 ehsm_gcm_cipher_test

测试 AEAD 的 GCM 模式加解密命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0102
- Process_Mode: one-pass
- Cipher_Direction: Cipher_enc
- Algorithm: AES-128

4.4.2.3 ehsm_ccm_cipher_test

测试 AEAD 的 CCM 模式加解密命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0103
- Process_Mode: one-pass
- Cipher_Direction: Cipher_enc
- Algorithm: SM4

4.4.2.4 ehsm_gmac_gen_test

测试 MAC 生成命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0201
- Process_Mode: one-pass
- MAC_Direction: Mac_gen
- Algorithm: SM4

4.4.2.5 ehsm_mac_gen_test

测试 MAC 生成命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0201
- Process_Mode: one-pass
- MAC_Direction: Mac_gen
- Algorithm: AES-128

4.4.3 PKE TB case

4.4.3.1 ehsm_pke_sm2_test

测试 SM2 解密算法命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0401
- Cipher_Direction: Cipher_dec

4.4.3.2 ehsm_pke_rsa_test

测试 RSA 加密算法命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0501
- Cipher_Direction: Cipher_enc

4.4.3.3 ehsm_rsa_sign_test

测试 RSA 签名命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0502
- Process_Mode: one-pass
- Sig_Direction: sig_gen
- Algorithm: SHA-1

4.4.3.4 ehsm_ecdsa_verify_test

测试 ECDSA 验签命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0601
- Process_Mode: one-pass
- Sig_Direction: sig_vry
- Algorithm: SHA-256

4.4.3.5 ehsm_sm2_verify

测试 SM 验签命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0x0402

- Process_Mode: one-pass
- Sig_Direction: sig_vry

4.4.4 TRNG TB case

4.4.4.1 ehsm_trng_test

测试随机数生成命令，Host 配置以下命令结构体，等待 eHSM 计算结束，获取随机数。

- Cmd_id: 0x0a01
- Post_Processing Algorithm: SM4 CTRDRBG

4.4.5 扩展服务 TB case

4.4.5.1 ehsm_get_challenge

测试获取挑战值命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0xff03
- eHSM_Debug_auth_Type: ehsm_challenge_type_ehsm_debug

4.4.5.2 ehsm_debug_auth_test

测试调试身份验证命令，Host 配置以下命令结构体，等待 eHSM 计算结束，校验计算结果正确性。

- Cmd_id: 0xff04
- eHSM_Debug_auth_Type: ehsm_challenge_type_ehsm_debug
- Algorithm: SM3-SM2

4.4.6 PQC TB case

4.4.6.1 ehsm_pqc_mlkem_keygen

测试 PQC MK-KEM-512 参数集的密钥生成功能。Host 配置以下命令结构体，触发 PQC 开始计算，轮询等待 PQC 计算结束，校验计算结果正确性。

- CFG_CMD: 0x1022c000
- Algorithm_Mode: MK_KEM.keygen

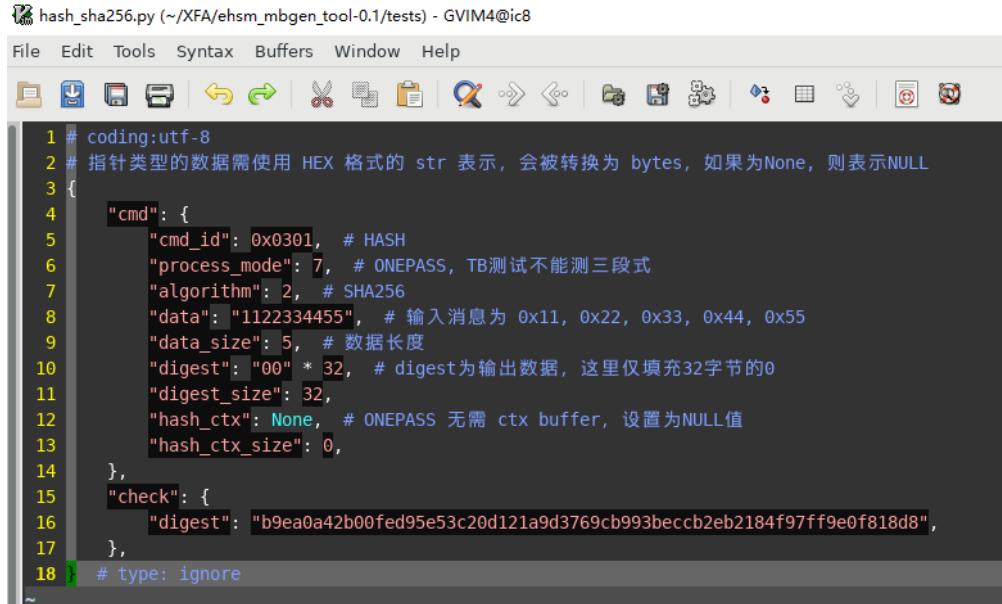
4.4.6.2 ehsm_pqc_mlkem_enc

测试 PQC MK-KEM-512 参数集的编码功能。Host 配置以下命令结构体，触发 PQC 开始计算，轮询等待 PQC 计算结束，校验计算结果正确性。

- CFG_CMD: 0x1122c000
- Algorithm_Mode: MK_KEM.enc

5 客户如何自己构造 case

1. 参照 [OSR_eHSM_Firmware_TRM_vx.x.pdf](#) 的 Mailbox 命令章节, 修改/ehsm_mbgen_tool-0.1/tests/目录下的 python 文件的“cmd”和“check”对应的字段, 配置算法和修改输入数据。“check”字段对应的是放在 hex 文件中 [Expect data addr](#) 的 golden 参考输出结果, 有些 case 有 golden 输出结果, 比如 hash_sha256, 有些没有 golden 输出结果, 比如 trng_test 和 import_xxx_key。



```

1 # coding:utf-8
2 # 指针类型的数据需使用 HEX 格式的 str 表示, 会被转换为 bytes, 如果为None, 则表示NULL
3 {
4     "cmd": {
5         "cmd_id": 0x0301, # HASH
6         "process_mode": 7, # ONEPASS, TB测试不能测三段式
7         "algorithm": 2, # SHA256
8         "data": "1122334455", # 输入消息为 0x11, 0x22, 0x33, 0x44, 0x55
9         "data_size": 5, # 数据长度
10        "digest": "00" * 32, # digest为输出数据, 这里仅填充32字节的0
11        "digest_size": 32,
12        "hash_ctx": None, # ONEPASS 无需 ctx buffer, 设置为NULL值
13        "hash_ctx_size": 0,
14    },
15    "check": {
16        "digest": "b9ea0a42b00fed95e53c20d121a9d3769cb993becb2eb2184f97ff9e0f818d8",
17    },
18 } # type: ignore

```

Figure 10: python 文件示例

2. 在/ehsm_mbgen_tool-0.1/路径下执行以下命令

```
./mbgen --group <fw|bl> test --base <addr> <test file> -o <output name>
```

其中:

--group <fw|bl>指定测试命令对应的组, fw 为 FW 的命令, bl 为 BL 的命令

--base <addr> 表示生成的数据在 RAM 中的基地址, 请写自己 SOC 对应的 SOC_MEM 的基地址

<test file> 表示测试数据文件, 例如 tests/hash_sha256.py

<output name> 表示输出的文件名, 例如 -o test_xxx 会生成两个文件: test_xxx.c 和 test_xxx.hex

test_xxx.c 文件中数组和 test_xxx.hex 等价, test_xxx.c 文件中包含测试的操作步骤和各个字段的地址、注释, 比 hex 文件的可读性更高。

3. 上一步生成出来的 test_xxx.hex 第一行会有命令行输入的 SOC_MEM 的地址, 删除第一行剩下的全部是待写入的数据, 然后将 test_xxx.hex 文件拷贝到 sim/tc/case/hex 路径下;

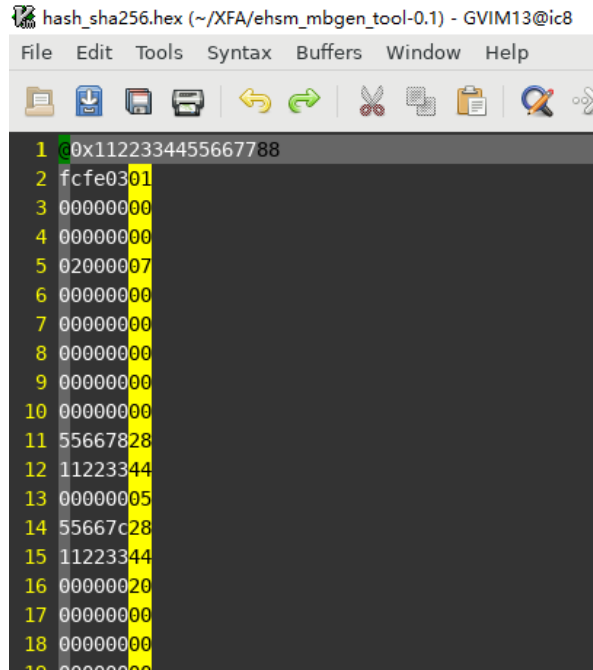


Figure 11: hex 文件示例

假如新构造的 case 名为 new_case.sv, 仿照已有的 case 修改 general_task() 的输入参数, 第一个参数改成自己 SOC 对应的 SOC_MEM 的基地址, 第二个参数改成所生成的 test_xxx.hex 的相对路径, 第三个参数改为第 2 步中的 output name 即可, 第四个参数不需要修改。

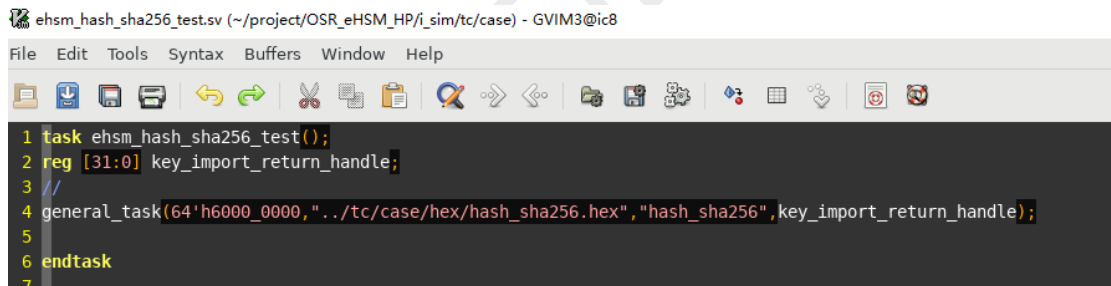


Figure 12: 调用 general_task 构造 case 示例

将 new_case.sv 的 task 添加到 sim/tb/tb.sv 的如下图位置, 以及将 new_case.sv 的路径添加到 sim/tc/tc.sv 的如下图位置中。

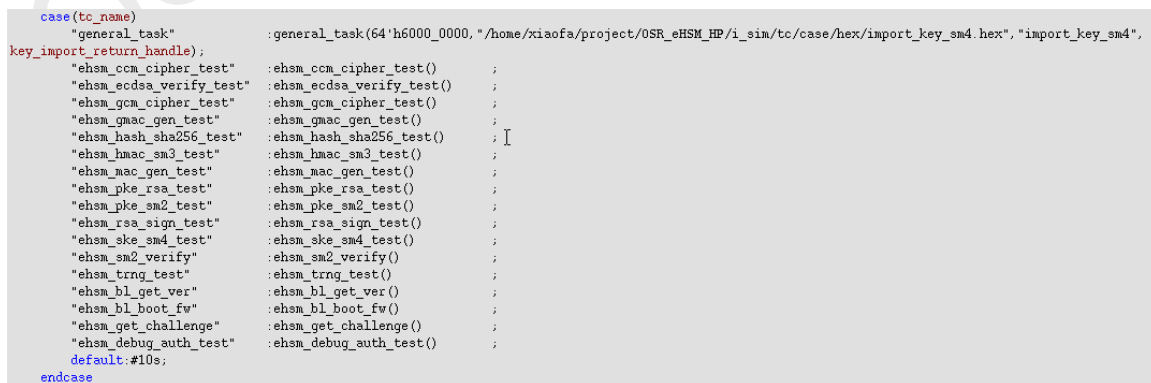


Figure 13: 添加 case name 到 tb.sv 的位置

```
`include "/case/ehsm_hash_sha256_test.sv";  
`include "/case/ehsm_hmac_sm3_test.sv";  
`include "/case/ehsm_ske_sm4_test.sv";  
`include "/case/ehsm_trng_test.sv";  
`include "/case/ehsm_rsa_sign_test.sv";  
`include "/case/ehsm_ecdsa_verify_test.sv";  
`include "/case/ehsm_gcm_cipher_test.sv";  
`include "/case/ehsm_ccm_cipher_test.sv";  
`include "/case/ehsm_gmac_gen_test.sv";  
`include "/case/ehsm_mac_gen_test.sv";  
`include "/case/ehsm_pke_sm2_test.sv";  
`include "/case/ehsm_pke_rsa_test.sv";  
`include "/case/ehsm_sm2_verify.sv";  
`include "/case/general_task.sv";  
`include "/case/ehsm_bl_get_ver.sv";  
`include "/case/ehsm_bl_boot_fw.sv";  
`include "/case/ehsm_get_challenge.sv";  
`include "/case/ehsm_debug_auth_test.sv";
```

Figure 14: 添加 case 路径到 tc.sv 的位置

6. 做完以上步骤之后, 在/sim 路径下, 执行如下命令即可运行新构造的 case

```
make sim waveswitch=on wavetime=0 TC=new_case
```

6 romcode 使用介绍

OSR eHSM 交付件提供了示例仿真环境所需的 romcode 文件，存放在 romcode/目录下，romcode 目录的内容如下表所示。

目录	文件名	说明
romcode/	ehsm_bootloader.romcode (ecc/cipher)	当次交付版本对应的 bootloader 固件
	ehsm_firmware_secure_boot_aes128.romcode	当次交付版本对应的带有安全启动信息的 firmware 固件
	mk_romcode_bin	mk_romcode 脚本，用于将 bin 文件转为 romcode 文件
	mk_romcode_hex	mk_romcode 脚本，用于将 hex 文件转为 romcode 文件

注意：

1. 固件是否带有 ecc 或 cipher 取决于实际的项目配置
2. 在 eHSM 的示例仿真环境中，OTP 配置字段 FW Control-eHSM[35:32]默认指定了 AES128 算法，因此提供的带安全启动信息的 firmware romcode 示例也由 aes128-cmac 算法加密/签名产生。若用户在本环境需支持其他算法，可使用 OSR 提供的 ehsm_image_tool 工具生成对应的 bin 文件，进而生成 romcode 文件

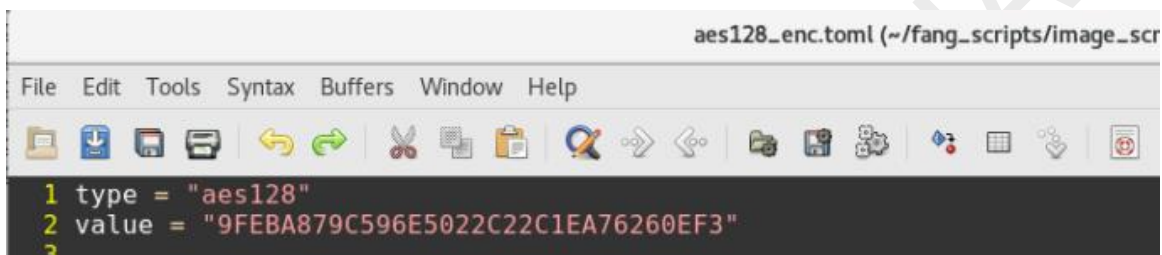
6.1 imgtool 工具介绍

imgtool 工具为 OSR 标准交付件内容，一般由 [sw/tools/imgtool-0.1.1.zip](#) 文件解压得到，可用于生成对应的包含加密/签名镜像头的二进制文件，使用时需由用户指定本地密钥及算法类型，关于 imgtool 工具相关的说明，请参考该工具的 user_guide.pdf（可由 imgtool-0.1.1.zip 文件解压得到）。

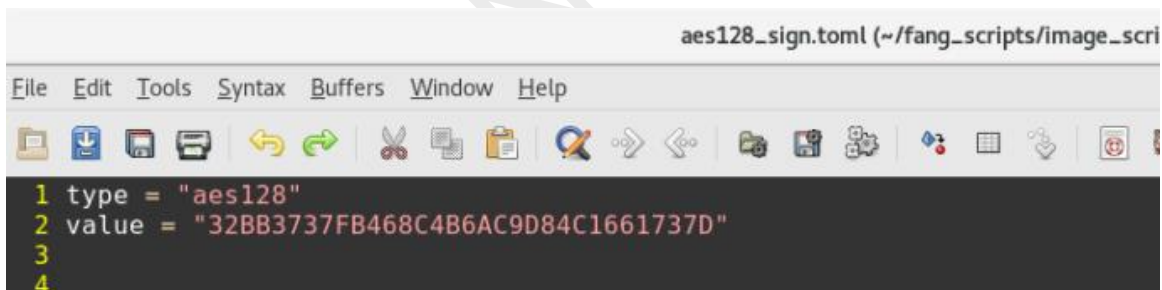
需提前准备如下文件，其中 ehsm_fw.bin 为原始 ehsm_firmware 固件

```
/home/fangjz/fang_scripts/image_script_v2/gen_secure_boot_image>ll
total 1632
-rw-rw-r-- 1 fangjz fangjz      59 May 18 14:16 aes128_enc.toml
-rw-rw-r-- 1 fangjz fangjz      59 May 18 14:16 aes128_sign.toml
-rw-rw-r-- 1 fangjz fangjz    256 May 18 14:16 boot_config.toml
-rw-rw-r-- 1 fangjz fangjz 145740 May 18 14:16 ehsm_fw.bin
-rwxr-xr-x 1 fangjz fangjz 1507992 May 18 14:16 imgtool
```

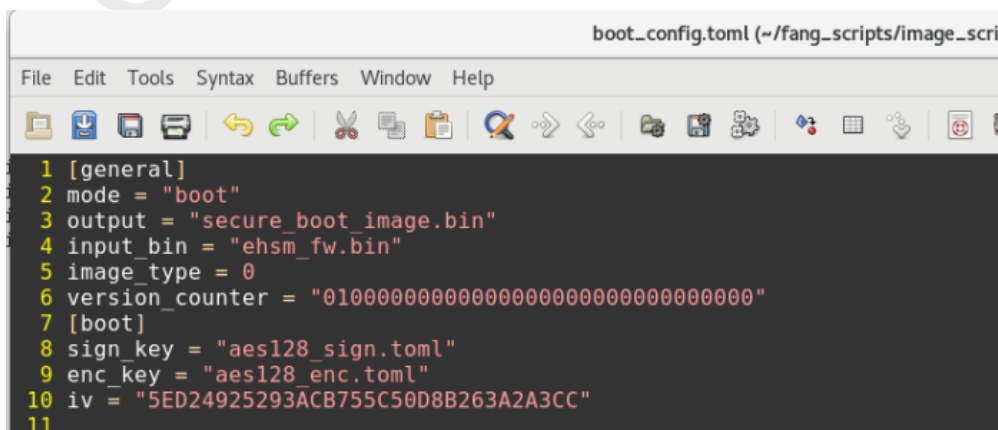
- (1) aes128_enc.toml 文件配置用于镜像加密的密钥和密钥类型，必须与步骤 1 中配置的 OTP 密钥值相同，且选择与 OTP 配置对应的安全启动算法。



- (2) aes128_sign.toml 文件配置用于镜像签名的密钥和密钥类型，必须与步骤 1 中配置的 OTP 密钥值相同，且选择与 OTP 配置对应的安全启动算法。



- (3) boot_config.toml 文件存放配置信息，用于描述镜像生成参数

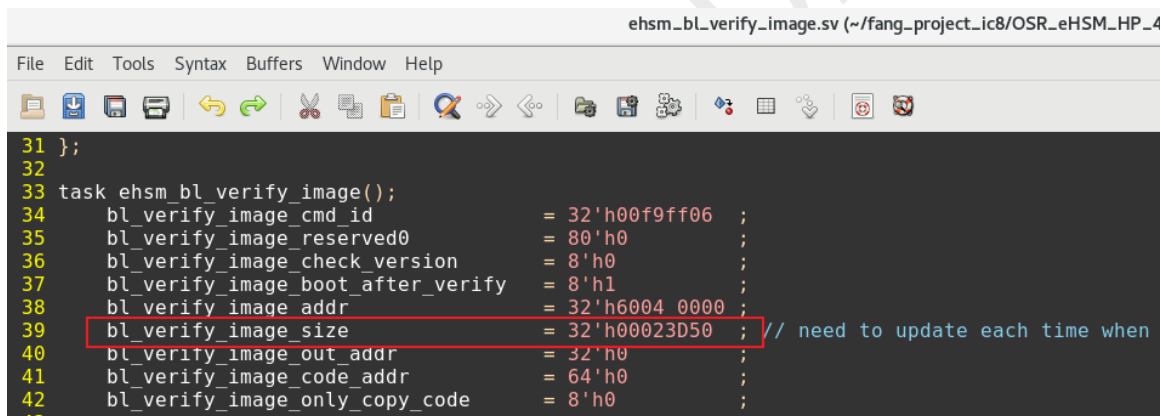


(4) 执行 `imgtool` 工具，带有头信息的加密固件 `secure_boot_image.bin`

```
/home/fangjz/fang_scripts/image_script_v2/gen_secure_boot_image>./imgtool boot_config.toml
Image generated successfully: secure_boot_image.bin (146768 bytes)
/home/fangjz/fang_scripts/image_script_v2/gen_secure_boot_image>
```

- 使用 `mk_romcode_bin` 工具处理 `secure_boot_image.bin` 文件，生成 `ehsm_firmware_secure_boot_aes128.romcode` 文件，替换 `romcode/ehsm_firmware_secure_boot_aes128.romcode` 文件。
- 手动替换 `sim/tc/case/ehsm_bl_verify_image.sv` 用例中的镜像大小字段，使其匹配带有头信息的加密固件（步骤 3 生成的 `secure_boot_image.bin`）的实际大小。

```
/home/fangjz/fang_scripts/image_script_v2/gen_secure_boot_image>ll
total 1776
-rw-rw-r-- 1 fangjz fangjz    64 May 18 14:22 aes128_enc.toml
-rw-rw-r-- 1 fangjz fangjz    62 May 18 14:24 aes128_sign.toml
-rw-rw-r-- 1 fangjz fangjz   260 May 18 14:25 boot_config.toml
-rw-rw-r-- 1 fangjz fangjz 145740 May 18 14:16 ehsm_fw.bin
-rwxr-xr-x 1 fangjz fangjz 1507992 May 18 14:16 imgtool
-rw-rw-r-- 1 fangjz fangjz 146768 May 18 14:29 secure_boot_image.bin
/home/fangjz/fang_scripts/image_script_v2/gen_secure_boot_image>stat -c "%s" secure_boot_image.bin | xargs printf "0x%X\n"
0x23D50
```



- 执行 `sim/run_regression` 进行回归，eHSM 用例通过，romcode 更新完成。

注：

- 以上示例中的指令和密钥算法类型仅对应 eHSM 交付时的仿真环境，可用于生成 OSR TB 环境中的仿真 bin 文件。
- 以上示例为采用 AES128 签名的固件，用户在实际使用该工具时，需结合本地的密钥和算法类型进行使用。
- 相对于原始的 `ehsm_fw.bin`，加密/签名后的镜像 `secure_boot_image.bin` 增加了 1KB 的头信息，即两者的大小存在如下关系（`bl_verify_image_size = code size + 0x400`）。